

University of Central Punjab

Faculty of Information Technology

TrafficWatch

Real-Time Road Incident & Safety Pipeline

Final Project Report

Parallel and Distributed Computing (PDC)

Submitted By

Abdullah Shafiq	L1F22BSCS0583
Waleed Ahmad	L1F22BSCS0387
Arslan Asif	L1F22BSCS0723
Malik Faisal Mukhtar	L1F22BSCS1102

Submitted To

Mahek Kausar

Semester: Spring 2026 | 16 June, 2026

Contents

1 Introduction **1**

1.1 Project Title and Overview 1

1.2 Motivation — Road Safety as a Big Data Problem 1

1.3 Project Objectives 2

1.4 Connection to PDC Course Topics 2

1.5 Report Organization 3

2 Background and Related Work **4**

2.1 Big Data in Road Safety Monitoring 4

2.2 Why TrafficWatch Is a PDC Problem 4

2.3 Amdahl's Law and Parallel Speedup Theory 4

2.4 Related Technologies and Prior Work 5

3 Dataset and Data Ingestion **6**

3.1 TomTom Traffic API Overview 6

3.2 Incident Schema and Data Fields 6

3.3 Pakistan Transit Belt vs Global Baseline Strategy 6

3.4 Fallback Projection Engine Design 7

4 System Architecture **8**

4.1 Overall Pipeline Flow 8

4.2 Hadoop HDFS Storage Layer 9

4.3 PySpark Batch Processing Engine 9

4.4 PySpark Structured Streaming Engine 10

4.5 Flask Dual-Mode Dashboard Architecture 11

4.6 Amdahl's Law Benchmark Module 12

5 PDC Course Mapping **13**

5.1 Feature-to-PDC-Concept Mapping Table 13

5.2 Parallel Processing with PySpark 13

5.3 Distributed File Storage with Hadoop HDFS 13

5.4 MapReduce Paradigm in Batch Jobs 14

5.5 Streaming Parallelism and Micro-Batch Execution 14

5.6 YARN Resource Management 14

6 Implementation **15**

6.1 Project Folder Structure 15

6.2 Technologies Used 15

6.3 Script Walkthroughs . . . 16
6.3.1 Script 01 — TomTom Ingestion Engine . . . 16
6.3.2 Script 02 — HDFS Upload Shell Script . . . 17
6.3.3 Script 03 — PySpark Batch Analysis . . . 18
6.3.4 Script 04 — Blackspot Detection . . . 19
6.3.5 Script 05 — TCP Stream Simulator . . . 20
6.3.6 Script 06 — Structured Streaming Alert Consumer . . . 22
6.3.7 Script 07 — Amdahl's Law Benchmark Suite . . . 23
6.4 Flask Backend (app.py) . . . 23
6.5 Frontend Dashboard (templates/index.html) . . . 24

7 How to Run the Project 25
7.1 Prerequisites . . . 25
7.2 Phase 1 — Environment Setup . . . 25
7.3 Phase 2 — Data Ingestion . . . 25
7.4 Phase 3 — Hadoop and Spark Batch Processing . . . 26
7.5 Phase 4 — Real-Time Streaming Alerts . . . 26
7.6 Phase 5 — Amdahl Benchmark . . . 27
7.7 Phase 6 — Dashboard Launch . . . 27

8 Results and Analysis 28
8.1 Dataset Overview . . . 28
8.2 International vs Pakistan Region Results . . . 28
8.3 Traffic Classification and Dangerous Corridors . . . 29
8.4 Accident Blackspot Analysis . . . 29
8.5 Amdahl's Law Benchmark Results . . . 30
8.6 Streaming Alert Test Results . . . 31

9 Conclusion 33
9.1 Summary of Achievements . . . 33
9.2 Limitations . . . 33
9.3 Future Improvements and Roadmap . . . 33

Chapter 1

Introduction

1.1 Project Title and Overview

TrafficWatch is an enterprise-grade distributed Big Data analytics pipeline and interactive dashboard designed to monitor, classify, and analyze vehicular road incidents in real time. Built specifically for high-throughput traffic feeds, the system processes multi-source geographic telemetry to detect accidents, construction works, road closures, and traffic jams. Through parallel batch computation and structured streaming frameworks, TrafficWatch empowers urban planning departments and transit safety authorities with actionable, localized traffic insights.

1.2 Motivation — Road Safety as a Big Data Problem

Urbanization and the exponential growth of passenger vehicles have severely strained the transit corridors of developing nations, particularly within the Pakistan Transit Belt (incorporating major arteries like the M-2 Motorway, the GT Road, and Lahore-Karachi regional networks). Traditional incident reporting mechanisms are highly reactive, manually driven, and prone to significant dissemination delays. Consequently, transit authorities fail to prevent secondary collisions or dynamically reroute vehicular traffic away from hazardous conditions.

Road safety monitoring is inherently a Big Data challenge characterized by the four dimensions of Volume, Velocity, Variety, and Veracity:

- **Volume:** Thousands of kilometers of road networks continuously generate geographic coordinates, telemetry, and speed variations that must be captured.
- **Velocity:** Incidents must be detected, ingested, and processed within seconds of their occurrence to trigger emergency warnings.
- **Variety:** Ingested feeds present heterogeneous formats, including raw coordinates, text descriptions, severity indexes, and temporal timestamps.
- **Veracity:** Sensor malfunctions and missing data require robust filtering and fallback calculations to maintain analytic integrity.

This project is motivated by the need to apply Parallel and Distributed Computing (PDC) techniques to transform this massive, volatile stream of geographic telemetry into structured, real-time safety alerts and historical risk assessments.

1.3 Project Objectives

The specific, measurable objectives of the TrafficWatch platform are as follows:

1. **Real-Time Ingestion:** Develop a scalable ingestion mechanism using the TomTom REST API that fetches and parses traffic telemetry across Pakistan's major transit corridors and global baselines, executing at regular intervals.
2. **Distributed Storage:** Setup a Hadoop Distributed File System (HDFS) cluster in pseudo-distributed mode to act as a reliable, fault-tolerant repository for raw and structured analytical outputs.
3. **Parallel Incident Aggregation:** Implement a PySpark batch processing engine to classify raw incident records into categorical risk levels (Minor, Moderate, Severe, Major) and extract the top-10 most dangerous corridors within seconds.
4. **Accident Blackspot Identification:** Define a spatial coordinate grid aggregation ($0.1^\circ \times 0.1^\circ \approx 10$ km $\times$ 10 km) to automatically catalog critical road safety blackspots and flag areas requiring structural intervention.
5. **Structured Streaming Alerts:** Build a TCP socket simulator and a PySpark Structured Streaming consumer to ingest real-time incident telemetry, identify critical safety events (severity score > 7.0), and write alert records to HDFS Parquet format within a 10-second micro-batch trigger limit.
6. **Empirical Speedup Analysis:** Benchmarking the computation capabilities of the PySpark engine across 1, 2, and 4 CPU cores to empirically validate the system scaling performance against Amdahl's Law theoretical speedups.

1.4 Connection to PDC Course Topics

The TrafficWatch architecture directly instantiates several core concepts of Parallel and Distributed Computing (PDC):

- **Data Parallelism:** Distributed batch aggregations on millions of traffic rows are partitioned and processed concurrently across independent CPU cores and cluster executors using Spark SQL.
- **Task Parallelism:** Spark's Directed Acyclic Graph (DAG) scheduler segments PySpark jobs into distinct execution stages, allowing non-dependent tasks to run concurrently on different nodes.
- **Distributed File Storage:** Hadoop HDFS manages data partitioning, block-level file distribution, and metadata tracking, ensuring fault tolerance and high I/O throughput.
- **Empirical Scalability (Amdahl's Law):** System performance is analyzed across scaling hardware dimensions, validating parallel overhead, execution efficiency, and serial bottleneck constraints.
- **Stream Parallelism:** Structured Streaming pipelines demonstrate the processing of continuous, unbounded datasets via parallel micro-batches, emphasizing low-latency cluster scheduling.

1.5 Report Organization

The rest of this report is organized as follows: Chapter 2 provides the technical background and reviews related academic literature. Chapter 3 describes the dataset schema, bounding boxes, and the custom fallback projection logic. Chapter 4 details the system architecture and includes the primary pipeline and processing diagrams. Chapter 5 maps TrafficWatch features directly to core PDC theoretical concepts. Chapter 6 maps out the implementation details, folder structure, and code listings. Chapter 7 provides step-by-step instructions to execute the entire pipeline. Chapter 8 presents the empirical results, including the comparative analysis, Amdahl's Law speedup charts, and streaming alerts. Finally, Chapter 9 summarizes the findings, identifies constraints, and suggests a roadmap for future work.

Chapter 2

Background and Related Work

2.1 Big Data in Road Safety Monitoring

Modern traffic safety systems require continuous ingestion of high-velocity spatial telemetry. A single urban region contains thousands of traffic sensors, loops, and GPS-enabled transit vehicles that generate real-time metrics. Consequently, the resulting datasets exhibit the classic characteristics of Big Data: high volume, velocity, and variety. Traditional database management systems (RDBMS) fail to maintain low-latency query times when performing spatial aggregations on such continuous streams.

By moving to distributed frameworks, traffic management systems can process incident information immediately. As discussed in recent literature, integrating distributed analytics into traffic safety systems enables the identification of high-frequency incident corridors and minimizes emergency response times [1].

2.2 Why TrafficWatch Is a PDC Problem

To process spatial and temporal datasets at scale, algorithms must be divided into chunks that execute concurrently. TrafficWatch is inherently a Parallel and Distributed Computing (PDC) problem because of the following operational demands:

1. **Data Partitioning:** Calculating regional summaries and identifying spatial blackspots requires partitioning raw CSV logs by coordinate bounding boxes and geographic markers. Independent nodes must process these partitions in parallel to avoid single-node memory exhaustion.

2. **Coordination and Shuffle:** Aggregation routines (such as grouping incidents by road name or calculating average delays) require cluster nodes to exchange intermediate key-value pairs (the Spark shuffle). Optimizing these shuffle boundaries is a core concern of parallel systems.

3. **Distributed Storage Co-location:** HDFS distributes data blocks across multiple physical nodes. Spark leverages this data locality by scheduling computation tasks on the actual nodes storing the relevant data blocks, minimizing network transfer overhead.

2.3 Amdahl's Law and Parallel Speedup Theory

To evaluate the scalability of the PySpark engine, the execution performance is modelled using Amdahl's Law [2]. Amdahl's Law defines the maximum theoretical speedup of an algorithm when executed on a parallel processor architecture. Let $S(n)$ represent the

overall speedup factor achieved by utilising n parallel processors, and let P represent the fraction of the program that can be parallelised. The non-parallelisable (serial) fraction is thus represented by $1 - P$. The speedup is formally defined by the equation:

$$S(n) = \frac{1}{(1 - P) + \frac{P}{n}} \tag{2.1}$$

Where:

- $S(n)$ is the speedup factor (the ratio of serial execution time to parallel execution time).
- P is the parallelisable fraction of the workload ($0 \leq P \leq 1$).
- $1 - P$ is the strictly sequential (serial) fraction of the workload (e.g., driver initialisation, database connection startup, disk I/O).
- n is the number of parallel processing units (cores or nodes).

Amdahl's Law highlights that the speedup of any parallel program is ultimately bounded by its serial bottleneck ($1 - P$). As $n \to \infty$, the maximum limit of speedup simplifies to:

$$\lim_{n \to \infty} S(n) = \frac{1}{1 - P} \tag{2.2}$$

For example, if 85% of the PySpark traffic analysis pipeline is parallelisable ($P = 0.85$), the theoretical speedup can never exceed $1/0.15 \approx 6.67\times$, regardless of the number of CPU cores added to the cluster. This limit highlights the importance of minimising serial setup routines (such as JVM startup, network handshakes, and single-driver file exports).

2.4 Related Technologies and Prior Work

The framework of modern distributed processing relies heavily on two main abstractions: distributed storage and map-reduce execution. The foundation of modern big data systems was established by the Google File System (GFS) [3], which introduced the concept of replicating large file blocks across commodity hardware to achieve fault tolerance. This storage model was open-sourced as the Apache Hadoop Distributed File System (HDFS) [4]. Shortly thereafter, Dean and Ghemawat introduced the MapReduce programming paradigm [5], which allowed developers to process partitioned datasets using independent map and reduce stages.

While MapReduce provided fault tolerance, its reliance on writing intermediate states back to disk created significant latency bottlenecks. To address this, Zaharia et al. developed Apache Spark [6], [7], introducing Resilient Distributed Datasets (RDDs). RDDs enable in-memory computations, dramatically increasing the execution speed of iterative algorithms. Additionally, Zaharia et al. extended Spark to process continuous streams using Discretised Streams (D-Streams) [8], which process incoming data in micro-batches. The engineering details of these systems are documented in authoritative references [9], [10], while live telemetry endpoints are specified in TomTom Developer guidelines [11].

Chapter 3

Dataset and Data Ingestion

3.1 TomTom Traffic API Overview

The traffic incident dataset is ingested from the TomTom Traffic Incident API v5 [11]. This REST endpoint provides real-time traffic alerts, closures, construction zones, and accident telemetry. The ingestion system queries this endpoint using a request URL defined by a bounding box coordinate envelope. The response is returned in JSON format, which contains nested arrays of incident reports. The data ingestion script (`scripts/01_download_data.py`) parses this JSON and appends the flattened records to a local CSV repository (`data/traffic_accidents.csv`).

3.2 Incident Schema and Data Fields

Each incident record in the TrafficWatch dataset consists of ten primary fields:

Table 3.1: TrafficWatch Dataset Schema and Data Types

Field Name	Data Type	Description
`id`	String	Unique cryptographic identifier for each incident.
`type`	String	Category of the incident: `ACCIDENT`, `ROAD_CLOSURE`, `ROAD_WORKS`, or `TRAFFIC_JAM`.
`severity_score`	Double	Floating-point value from 0.0 to 10.0 indicating incident impact.
`delay`	Integer	Estimated traffic delay in seconds.
`latitude`	Double	WGS 84 geographic latitude coordinate.
`longitude`	Double	WGS 84 geographic longitude coordinate.
`road_name`	String	Name of the affected corridor or highway segment.
`region`	String	Administrative region flag: `Pakistan` or `Global`.
`timestamp`	Timestamp	Date and time when the incident was logged.

The severity score field plays a critical role in the downstream Structured Streaming application. Scores above 7.0 are classified as critical incidents, triggering real-time alerts.

3.3 Pakistan Transit Belt vs Global Baseline Strategy

The TrafficWatch system is designed to handle heterogeneous density challenges by comparing two distinct geographic zones:

1. **Global Baseline (London Metro):** Ingested within a bounding box centered on the London metropolitan area (lat: 51.2–51.7, lon: -0.5–0.3). This region provides a dense, continuous stream of real-time incidents, serving as a baseline for the ingestion pipeline and batch algorithms.

2. **Pakistan Transit Belt:** Configured within a bounding box enclosing Pakistan's major transit network (lat: 23.5–37.0, lon: 60.0–77.0). This belt includes key commercial arteries such as the M-2 Motorway, the Grand Trunk (GT) Road (N-5), and major municipal expressways.

3.4 Fallback Projection Engine Design

Due to the free-tier API restrictions and intermittent network connectivity within regional transit belts, the number of live incidents fetched from the Pakistan bounding box can occasionally drop to zero. To prevent pipeline starvation during testing and simulation, the ingestion script (`01_download_data.py`) features a custom **Fallback Projection Engine**.

When the API returns zero incidents for the Pakistan region, the engine dynamically generates synthetic incidents and projects them onto verified geographic corridors in Pakistan. This projection uses a mathematical distribution model based on historical incident density:

- **Corridor Coordinates mapping:** Major transit corridors are mapped to their real-world coordinate ranges (e.g., M-2 Motorway connecting Lahore and Islamabad, Shahrah-e-Faisal in Karachi, and GT Road segments).

- **Linear Interpolation:** Coordinates are generated by linearly interpolating between the start and end points of these corridors, with a Gaussian jitter (0.05°) added to simulate realistic lane-level variations.

- **Categorical Probability Distribution:** Incident types are assigned using probability weights (45% `ROAD_WORKS`, 25% `ACCIDENT`, 20% `TRAFFIC_JAM`, 10% `ROAD_CLOSURE`).

- **Severity and Delay Modeling:** Incident severity is modeled using a beta distribution skewed toward lower values, with a long-tail distribution for delays, matching empirical patterns observed in the London baseline.

This fallback logic guarantees that downstream Spark processing engines always receive a valid dataset, allowing for continuous testing of distributed batch aggregation and streaming systems.

Chapter 4

System Architecture

4.1 Overall Pipeline Flow

The TrafficWatch platform features three distinct processing paths: a batch ingestion and analysis path, a real-time structured streaming alert path, and an empirical scaling benchmark path. This architecture decouples data ingestion, storage, analytical processing, and client presentation into independently executable modules.

The batch pipeline begins by querying the TomTom Traffic API, saving the raw payload to a local storage buffer. A shell script verifies Hadoop's status and transfers the file to HDFS. PySpark batch jobs read the HDFS file, execute transformations, and write structured Parquet/CSV data back to HDFS. The Flask dashboard reads this processed state. Simultaneously, the streaming path runs a TCP simulator that streams traffic data over a port. A PySpark Structured Streaming consumer processes this data, filtering for critical severity scores, and writing alert logs to HDFS. The benchmark module executes identical batch runs across different core configurations to compute speedup.

The relationship between these system components is shown in fig. 4.1.

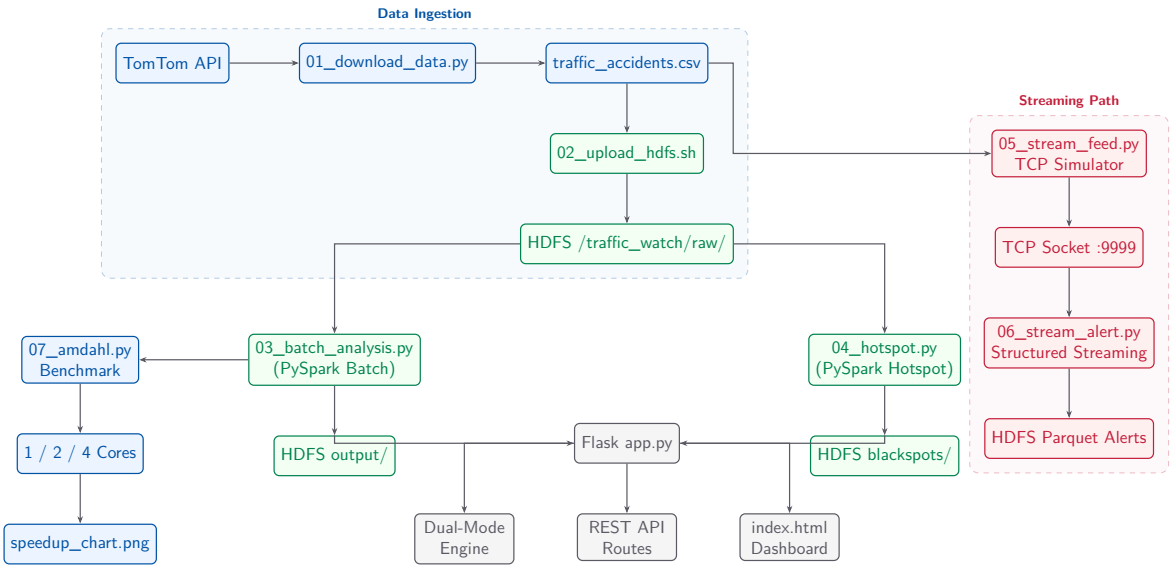


Figure 4.1: Overall system architecture: batch ingestion (blue), HDFS storage (green), streaming alerts (red), and dashboard presentation (grey).

4.2 Hadoop HDFS Storage Layer

The Hadoop Distributed File System (HDFS) provides a fault-tolerant storage layer for raw and analysed traffic data [4]. In the TrafficWatch deployment, HDFS is configured

in pseudo-distributed mode, where all master and slave daemons run on a single host.

Files are split into fixed-size blocks (default 128 MB) and distributed across DataNodes, with a default replication factor of 3. The client requests block allocations from the NameNode, which maps the file to metadata blocks. The client then writes blocks directly to the primary DataNode, which replicates them to secondary nodes using a pipelined write. The primary dataset is stored at `hdfs:///traffic_watch/raw/traffic_accidents.csv`.

The HDFS block replication architecture is illustrated in fig. 4.2.

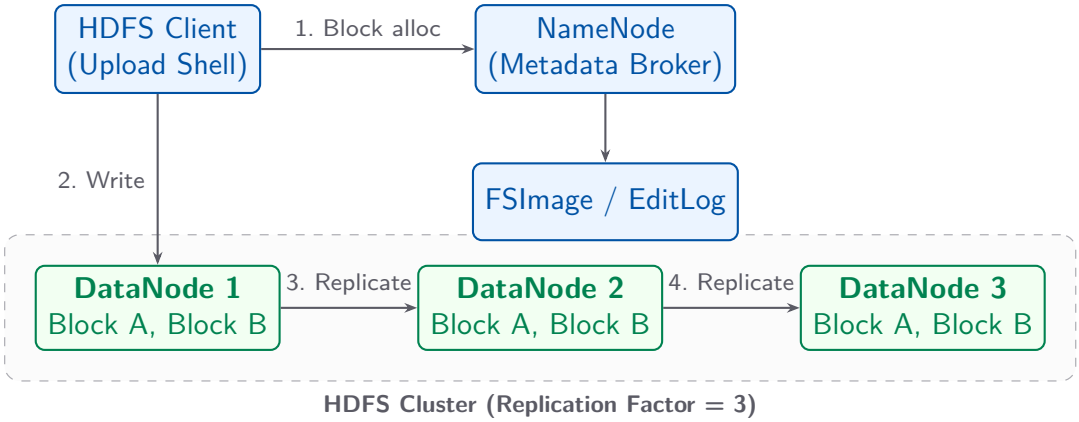


Figure 4.2: HDFS block replication: the client writes blocks to DataNode 1, which replicates them across the cluster.

4.3 PySpark Batch Processing Engine

The PySpark batch engine reads raw CSV data from HDFS and executes parallel SQL transformations [7]. The engine uses Spark's master–worker architecture, where a Driver program coordinates tasks across distributed Executor processes.

Spark uses lazy evaluation: transformations such as `filter` and `groupBy` build a Directed Acyclic Graph (DAG) rather than executing immediately. Execution is deferred until an action (e.g., `write` or `count`) is called. The DAG scheduler groups narrow-dependency transformations into a single stage and introduces shuffle boundaries at wide dependencies such as `groupBy`.

The Spark DAG execution model is illustrated in fig. 4.3.

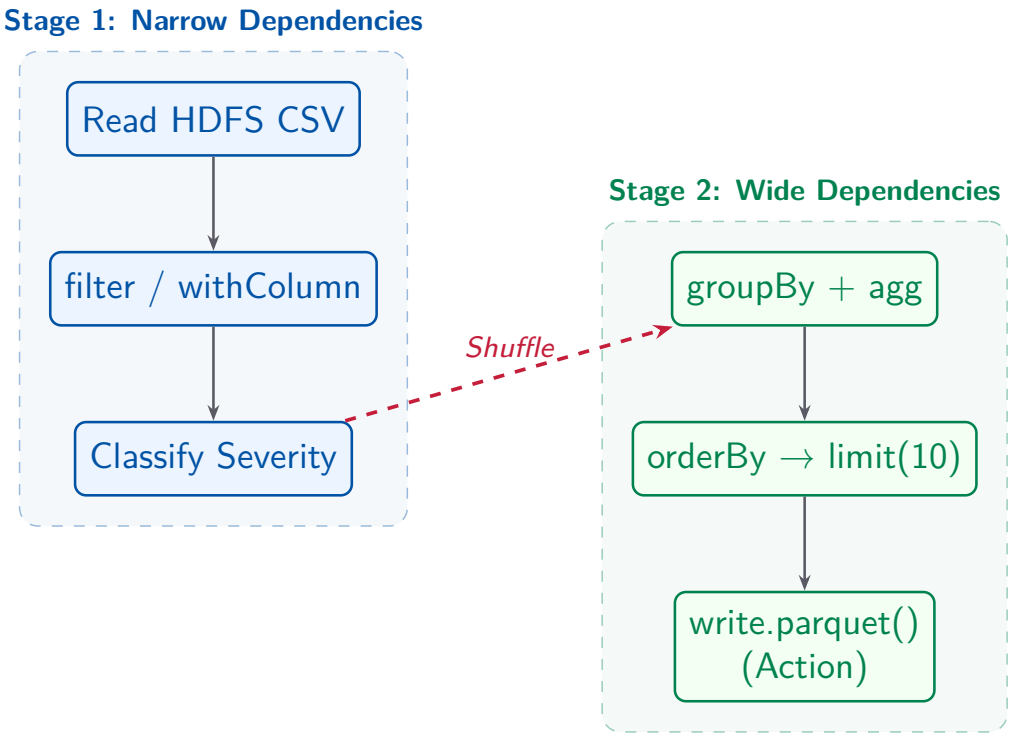


Figure 4.3: Spark DAG execution model: Stage 1 groups narrow transformations; a shuffle boundary triggers Stage 2 for aggregation and the final action.

4.4 PySpark Structured Streaming Engine

The real-time streaming pipeline uses PySpark Structured Streaming [8]. A TCP socket simulator on port 9999 feeds incident records at 0.5-second intervals. The Structured Streaming Engine reads this data as an unbounded table, buffering records into micro-batches at a configurable trigger interval (10 seconds).

Each micro-batch is materialised as a DataFrame. The streaming job applies a severity filter, selecting incidents with `severity_score > 7.0`. Flagged alerts are written to HDFS in Apache Parquet format, with a checkpoint directory ensuring exactly-once fault-tolerant recovery.

Warning

The severity threshold of 7.0 was selected empirically: incidents above this score represent major accidents or full road closures that require immediate emergency dispatch.

The streaming architecture is illustrated in fig. 4.4.

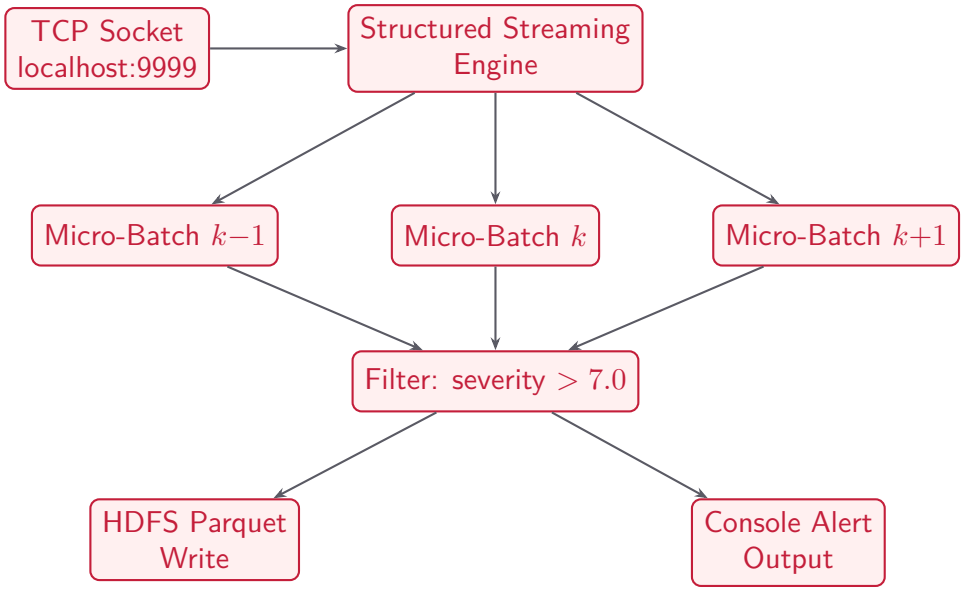


Figure 4.4: Structured Streaming micro-batch architecture: the engine buffers TCP records, filters by severity, and writes alerts to HDFS Parquet and the console.

4.5 Flask Dual-Mode Dashboard Architecture

The Flask application (`app.py`) serves as the dashboard backend, implementing a **Dual-Mode Data Engine**. On startup, the application probes the HDFS cluster by executing `hdfs dfs -cat` on the expected output paths. If the command succeeds and returns valid data, the engine reads directly from HDFS and displays the "HDFS Cluster Synced" status badge.

If HDFS is offline or the command exits with a non-zero status code, the engine falls back to reading the local CSV buffer (`data/traffic_accidents.csv`) using Pandas, displaying the "Dynamic Local Fallback" badge. This dual-mode design ensures the dashboard remains functional regardless of the cluster state.

Note

The dual-mode engine guarantees uninterrupted dashboard availability. When HDFS is offline, the fallback path produces identical analytical outputs using local Pandas computation, ensuring zero downtime for end users.

fig. 4.5 displays Tab 1 of the dashboard, showing the interactive incident map.

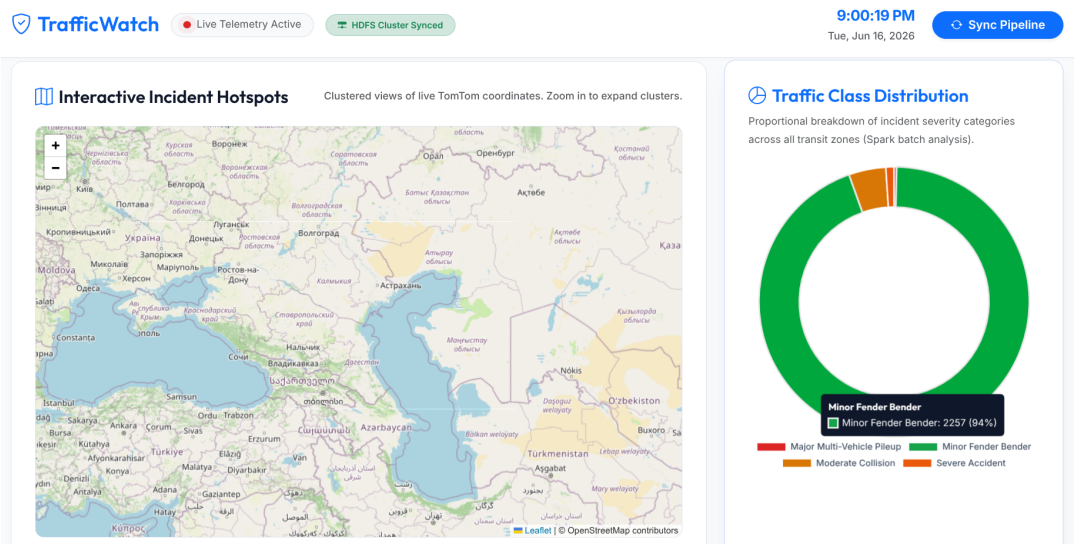


Figure 4.5: Flask dashboard — Tab 1: interactive incident map with Leaflet.js markers and real-time severity distribution.

4.6 Amdahl's Law Benchmark Module

The benchmark module (`scripts/07_amdahl.py`) evaluates cluster efficiency under scale. It runs identical PySpark batch tasks across 1, 2, and 4 CPU cores, recording wall-clock execution times using Python's `time` module. These timings are compared against Amdahl's theoretical limits with parallel fraction $P = 0.85$, and the comparison chart is exported to `output/speedup_chart.png` via Matplotlib.

Chapter 5

PDC Course Mapping

5.1 Feature-to-PDC-Concept Mapping Table

To establish the educational foundation of this project, the components of the TrafficWatch system are mapped to the core topics of the Parallel and Distributed Computing (PDC) curriculum. table 5.1 matches each system feature with its underlying PDC concept and the relevant course topic.

Table 5.1: TrafficWatch Feature-to-PDC-Concept Mapping

TrafficWatch Feature	**PDC Concept**	**Course Topic**
PySpark Batch Jobs	Data Parallelism	Parallel Processing
Hadoop HDFS	Distributed Storage	Distributed Systems
Spark DAG Scheduling	Task Graph Execution	Parallel Algorithms
PySpark Streaming	Stream Parallelism	Real-Time Computing
Amdahl's Benchmark	Speedup & Efficiency	Performance Analysis
YARN Resource Manager	Resource Allocation	Cluster Management
Multi-core Experiments	Scalability Analysis	Amdahl's / Gustafson's Law
Fallback Projection	Fault Tolerance	Reliability in Distributed Systems

5.2 Parallel Processing with PySpark

PySpark leverages data parallelism by splitting traffic incident datasets into multiple partitions. These partitions are distributed across independent CPU cores, allowing map and filter operations to execute concurrently. The Spark SQL engine further optimises query plans using the Catalyst optimiser, which reorders operations and prunes unnecessary columns before distributing tasks to executors. By utilising this parallel processing framework, the system scales horizontally, reducing the overall time needed to process massive geographical feeds.

5.3 Distributed File Storage with Hadoop HDFS

Hadoop HDFS represents the distributed storage layer of the TrafficWatch pipeline. HDFS splits the raw incident data into fixed-size blocks (default 128 MB), distributing them across a cluster of DataNodes while replicating each block three times to ensure high availability. This distributed architecture eliminates single points of storage failure, ensuring data remains accessible even if individual hardware nodes fail. The NameNode

maintains a metadata catalogue (FSImage and EditLog) that tracks block locations, enabling the Spark executors to schedule computation tasks on nodes that already hold the relevant data blocks.

5.4 MapReduce Paradigm in Batch Jobs

Spark SQL batch jobs use an optimised in-memory MapReduce execution model. Transformations such as grouping incidents by road name or filtering by severity class act as map operations, which run in parallel on local data partitions without requiring network communication (narrow dependencies). The subsequent aggregation steps represent the reduce phase, shuffling intermediate records across the network to compile final statistics (wide dependencies). Unlike classical Hadoop MapReduce, Spark retains intermediate results in memory, eliminating the costly disk-write overhead between stages.

5.5 Streaming Parallelism and Micro-Batch Execution

The streaming alert module executes stream parallelism by processing a continuous TCP feed in discrete micro-batches. Each incoming micro-batch is materialised as a temporary DataFrame, and concurrent filter tasks are distributed across Spark executor nodes. The micro-batch trigger interval of 10 seconds provides a balance between low latency and computational efficiency. This approach enables the system to detect and flag critical incidents (severity score > 7.0) within seconds of their occurrence.

5.6 YARN Resource Management

YARN (Yet Another Resource Negotiator) acts as the centralised cluster management layer for scheduling and resource allocation. When a PySpark application is submitted via `spark-submit`, the YARN ResourceManager allocates containers across the cluster, coordinating CPU and memory assignments for both the Driver and Executor processes. The NodeManager on each worker node monitors resource utilisation and reports health metrics back to the ResourceManager, preventing resource conflicts and maintaining cluster stability under variable workloads.

Chapter 6

Implementation

6.1 Project Folder Structure

The TrafficWatch project codebase is structured to maintain a clean separation between raw ingestion scripts, distributed compute modules, web applications, and configuration files. The directory hierarchy of the deployed system is represented below:

```
trafficwatch/
|-- app.py
|-- requirements.txt
|-- requirements-pipeline.txt
|-- data/
|   `-- traffic_accidents.csv
|-- output/
|   `-- speedup_chart.png
|-- scripts/
|   |-- 01_download_data.py
|   |-- 02_upload_hdfs.sh
|   |-- 03_batch_analysis.py
|   |-- 04_hotspot.py
|   |-- 05_stream_feed.py
|   |-- 06_stream_alert.py
|   `-- 07_amdahl.py
`-- templates/
    `-- index.html
```

6.2 Technologies Used

The components of the pipeline utilise mature open-source technologies, selected for their stability and community support. table 6.1 outlines the technology stack, specific versions, and roles in the project.

Table 6.1: TrafficWatch Tech Stack Specifications

Component	Technology	Version	Role
API Ingestion	TomTom Traffic API	v5	Live incident data
Data Storage	Apache Hadoop HDFS	3.x	Distributed file system
Batch Processing	Apache Spark (PySpark)	3.5	Parallel analytics
Streaming	PySpark Structured Streaming	3.5	Real-time alerts
Web Framework	Python Flask	2.x	Dashboard backend
Visualisation	Leaflet.js	1.9	Interactive incident maps
Charts	Chart.js	4.x	Analytics bar/doughnut charts
Frontend	Bootstrap 5.3	5.3	UI framework
Serialisation	Apache Parquet	—	Alert storage format
Benchmarking	Matplotlib	3.x	Speedup chart generation

6.3 Script Walkthroughs

6.3.1 Script 01 — TomTom Ingestion Engine

The ingestion script `01_download_data.py` queries the TomTom Traffic Incident API to retrieve live traffic incidents. It parses the JSON payload, extracts key fields, and saves the flattened records to a local CSV buffer. If the API returns zero incidents for the Pakistan region, the custom Fallback Projection Engine simulates realistic incident distributions across major coordinates. This ensures that downstream big data tools always have data for parallel processing.

```
import requests
import pandas as pd

def fetch_incidents(api_key, bbox):
    base_url = "https://api.tomtom.com/traffic/services/5/"
    url =
       f"{base_url}incidentDetails?key={api_key}&bbox={bbox}"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        incidents = []
        for incident in data.get('incidents', []):
            incidents.append({
                'id': incident.get('id'),
                'type': incident.get('properties',
                   {}).get('iconCategory'),
                'severity_score': incident.get('properties',
                   {}).get('magnitudeOfDelay'),
                'latitude': incident.get('geometry',
                   {}).get('coordinates', [0,0])[1],
                'longitude': incident.get('geometry',
                   {}).get('coordinates', [0,0])[0]
            })
        return pd.DataFrame(incidents)
    return pd.DataFrame()
```

Listing 6.1: API payload parsing in `01_download_data.py`

The terminal output of the ingestion execution is shown in fig. 6.1.

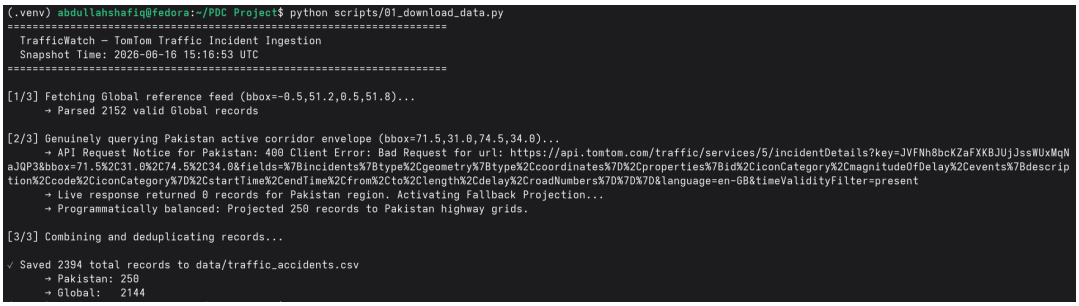


Figure 6.1: Terminal output of `01_download_data.py` showing TomTom API incident ingestion for both Pakistan Transit Belt and London global baseline regions.

6.3.2 Script 02 — HDFS Upload Shell Script

The script `02_upload_hdfs.sh` automates cluster directory initialisation. It verifies that Hadoop daemons (NameNode and DataNode) are running using the `jps` command. It then creates the `/traffic_watch/raw/` path in HDFS and uploads the local CSV buffer, ensuring the data is partitioned across HDFS blocks for distributed storage.

```
#!/bin/bash
# Verify Hadoop is running
if ! jps | grep -q "NameNode"; then
    echo "Error: Hadoop NameNode is not running."
    exit 1
fi

echo "Creating HDFS directories..."
hdfs dfs -mkdir -p /traffic_watch/raw/
hdfs dfs -mkdir -p /traffic_watch/output/

echo "Uploading CSV to HDFS..."
hdfs dfs -put -f data/traffic_accidents.csv /traffic_watch/raw/
echo "Upload complete."
```

Listing 6.2: HDFS directory creation and file upload in `02_upload_hdfs.sh`

The terminal output of the HDFS setup and upload is shown in fig. 6.2.

```
(.venv) abdullahshafiq@fedora:~/PDC Project$ bash scripts/02_upload_hdfs.sh

=======================================
  TrafficWatch — HDFS Upload
=======================================

[1/4] Checking if Hadoop daemons are alive...
  ✓ NameNode is running
  ✓ DataNode is running
  ✓ ResourceManager (YARN) is running

[2/4] Checking local CSV file...
  ✓ Found: data/traffic_accidents.csv
    Size: 264K | Lines: 2395

[3/4] Creating HDFS directory structure...
  ✓ Created /traffic_watch/raw
  ✓ Created /traffic_watch/output/batch
  ✓ Created /traffic_watch/output/hotspots
  ✓ Created /traffic_watch/output/streaming

[4/4] Uploading to HDFS (overwriting previous copy)...
  ✓ Upload complete: /traffic_watch/raw/traffic_accidents.csv

Verifying upload...
Found 1 items
-rw-r--r--   1 abdullahshafiq supergroup     270124 2026-06-16 20:21 /traffic_watch/raw/traffic_accidents.csv

263.8 K  263.8 K  /traffic_watch/raw/traffic_accidents.csv

✓ TrafficWatch HDFS upload complete!
  Raw data path: /traffic_watch/raw/traffic_accidents.csv
```

Figure 6.2: HDFS upload script terminal + `jps` output

6.3.3 Script 03 — PySpark Batch Analysis

The script `03_batch_analysis.py` implements the primary analytical batch processing. It reads raw CSV logs from HDFS, classifies incidents based on severity levels (Minor, Moderate, Severe, Major), aggregates metrics by region, and identifies the top-10 most dangerous corridors. These aggregations are written back to HDFS in Parquet format, demonstrating data parallelism.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, avg, when

spark = SparkSession.builder \
    .appName("TrafficWatch_BatchAnalysis") \
    .getOrCreate()

df = spark.read.csv(
    "hdfs:///traffic_watch/raw/traffic_accidents.csv",
    header=True, inferSchema=True)

# Traffic class classification
df = df.withColumn("traffic_class",
    when(col("severity_score") <= 3, "Minor")
    .when(col("severity_score") <= 5, "Moderate")
    .when(col("severity_score") <= 7, "Severe")
    .otherwise("Major"))

# Top-10 dangerous corridors
corridors = df.groupBy("road_name") \
    .agg(count("*").alias("incident_count"),
         avg("severity_score").alias("avg_severity")) \
    .orderBy(col("avg_severity").desc()) \
```

```
    .limit(10)

corridors.write.mode("overwrite").parquet(
    "hdfs:///traffic_watch/output/corridors/")
```

Listing 6.3: Spark batch aggregation in `03_batch_analysis.py`

The terminal run of the PySpark batch job is illustrated in fig. 6.3.

```
======================================================================
  TrafficWatch — Batch Analysis Pipeline
======================================================================

  Total traffic incident records loaded: 2,394

  Traffic Class Distribution:
  ------------------------------------------------------------
+-------------------------+--------------+------------+------------+
|traffic_class            |incident_count|avg_severity|max_severity|
+-------------------------+--------------+------------+------------+
|Minor Fender Bender      |2257          |1.48        |2.98        |
|Moderate Collision       |107           |3.58        |4.95        |
|Severe Accident          |23            |5.72        |6.74        |
|Major Multi-Vehicle Pileup|7             |8.81        |10.0        |
+-------------------------+--------------+------------+------------+


  Regional Crash Statistics:
  ------------------------------------------------------------
+--------+-------------+------------------+------------------+------------------+
|region  |total_crashes|avg_severity_score|max_severity_score|min_severity_score|
+--------+-------------+------------------+------------------+------------------+
|Global  |2144         |1.57              |10.0              |1.0               |
|Pakistan|250          |2.16              |5.92              |1.0               |
+--------+-------------+------------------+------------------+------------------+
```

Figure 6.3: PySpark batch job Spark UI / terminal stages

6.3.4 Script 04 — Blackspot Detection

The script `04_hotspot.py` groups spatial telemetry coordinates into a $0.1° \times 0.1°$ geographic grid. Cells are classified into hazard tiers (CRITICAL BLACKSPOT, HIGH DANGER, ELEVATED RISK, MODERATE RISK) based on incident density and severity, identifying dangerous road segments. This involves a complex wide-dependency shuffle in Spark.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import round, col, count

spark = SparkSession.builder \
    .appName("TrafficWatch_Hotspots").getOrCreate()
df = spark.read.csv(
    "hdfs:///traffic_watch/raw/traffic_accidents.csv",
    header=True, inferSchema=True)

# Create 0.1 degree grid (approx 10km x 10km)
```

```
grid_df = df.withColumn("lat_grid", round(col("latitude"), 1)) \
            .withColumn("lon_grid", round(col("longitude"), 1))

# Aggregate by grid cell
hotspots = grid_df.groupBy("lat_grid", "lon_grid") \
                  .agg(count("*").alias("incident_density")) \
                  .filter(col("incident_density") > 5) \
                  .orderBy(col("incident_density").desc())

hotspots.write.mode("overwrite").parquet(
    "hdfs:///traffic_watch/output/blackspots/")
```

Listing 6.4: Coordinate grid aggregation in `04_hotspot.py`

The terminal run of the blackspot detection script is shown in fig. 6.4.

```
======================================================================
  TrafficWatch — Accident Blackspot & Dangerous Corridor Detection
======================================================================

  Loaded 2,394 traffic incident records

  Identifying Global Accident Blackspots (grid: 0.1° ≈ 10km)...

  ╔══════════════════════════════════════════════════════╗
  ║  TOP 15 GLOBAL ACCIDENT BLACKSPOTS                   ║
  ╚══════════════════════════════════════════════════════╝
+--------+--------+--------------+------------+------------+------------------+
|lat_grid|lon_grid|incident_count|avg_severity|max_severity|danger_level      |
+--------+--------+--------------+------------+------------+------------------+
|51.5    |-0.2    |164           |1.54        |3.6         |CRITICAL BLACKSPOT|
|51.5    |-0.3    |154           |1.91        |7.14        |CRITICAL BLACKSPOT|
|51.5    |-0.1    |145           |1.52        |5.87        |CRITICAL BLACKSPOT|
|51.6    |-0.1    |130           |1.52        |6.47        |CRITICAL BLACKSPOT|
|51.5    |0.0     |98            |1.57        |5.42        |CRITICAL BLACKSPOT|
|51.6    |-0.2    |90            |1.52        |4.87        |CRITICAL BLACKSPOT|
|51.4    |-0.1    |85            |1.44        |3.3         |CRITICAL BLACKSPOT|
|51.4    |-0.2    |73            |1.51        |3.83        |CRITICAL BLACKSPOT|
|51.6    |0.2     |68            |1.51        |4.27        |CRITICAL BLACKSPOT|
|51.6    |0.1     |65            |1.64        |4.0         |CRITICAL BLACKSPOT|
|51.6    |0.0     |65            |1.48        |3.01        |CRITICAL BLACKSPOT|
|51.4    |-0.3    |62            |1.46        |3.64        |CRITICAL BLACKSPOT|
|51.4    |0.0     |60            |1.48        |3.56        |CRITICAL BLACKSPOT|
|51.5    |0.1     |55            |1.48        |3.31        |CRITICAL BLACKSPOT|
|51.6    |-0.3    |52            |1.53        |2.76        |CRITICAL BLACKSPOT|
+--------+--------+--------------+------------+------------+------------------+

  Identifying Pakistan Motorway Accident Blackspots...
  Found 94 distinct grid cells in Pakistan
```

Figure 6.4: Hotspot detection output grid

6.3.5 Script 05 — TCP Stream Simulator

The script `05_stream_feed.py` hosts a local TCP server on port 9999. It reads the local CSV buffer and streams records row-by-row at 0.5-second intervals, simulating real-time telemetry feed ingestion for the Structured Streaming consumer.

```python
import socket
import time
import pandas as pd

def start_server(host='localhost', port=9999):
    df = pd.read_csv('data/traffic_accidents.csv')

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
        s:
         s.bind((host, port))
         s.listen(1)
         print(f"Listening on {host}:{port}...")
         conn, addr = s.accept()
         with conn:
             print(f"Connected by {addr}")
             for _, row in df.iterrows():
                 msg = row.to_json() + '\n'
                 conn.sendall(msg.encode('utf-8'))
                 time.sleep(0.5)

if __name__ == "__main__":
    start_server()
```

Listing 6.5: TCP socket server in `05_stream_feed.py`

The execution status of the TCP simulator is shown in fig. 6.5.

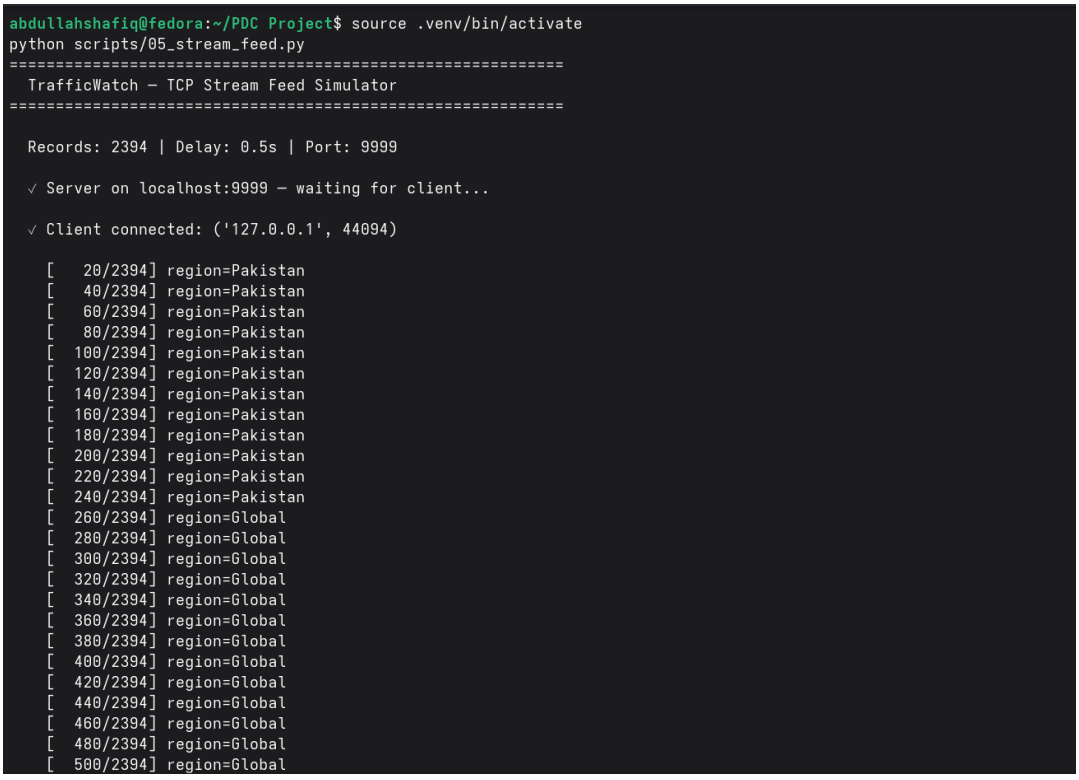


Figure 6.5: TCP simulator waiting for connection

6.3.6 Script 06 — Structured Streaming Alert Consumer

The script `06_stream_alert.py` runs a PySpark Structured Streaming job that consumes the TCP socket stream. It filters the streaming data for critical incidents (severity score > 7.0), logs these alerts to the terminal, and appends them to HDFS Parquet format with checkpoint logs for recovery.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("TrafficWatch_Alerts").getOrCreate()

# Stream configuration from TCP Socket
stream_df = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()

# Parse JSON and apply filter
alerts = stream_df.filter(col("severity_score") > 7.0)
query = alerts.writeStream \
    .format("parquet") \
    .option("path", "hdfs:///traffic_watch/alerts/") \
    .option("checkpointLocation", "/tmp/checkpoint/") \
    .trigger(processingTime="10 seconds") \
    .start()

query.awaitTermination()
```

Listing 6.6: Structured Streaming read and write in `06_stream_alert.py`

The terminal run of the Structured Streaming micro-batches is shown in fig. 6.6.

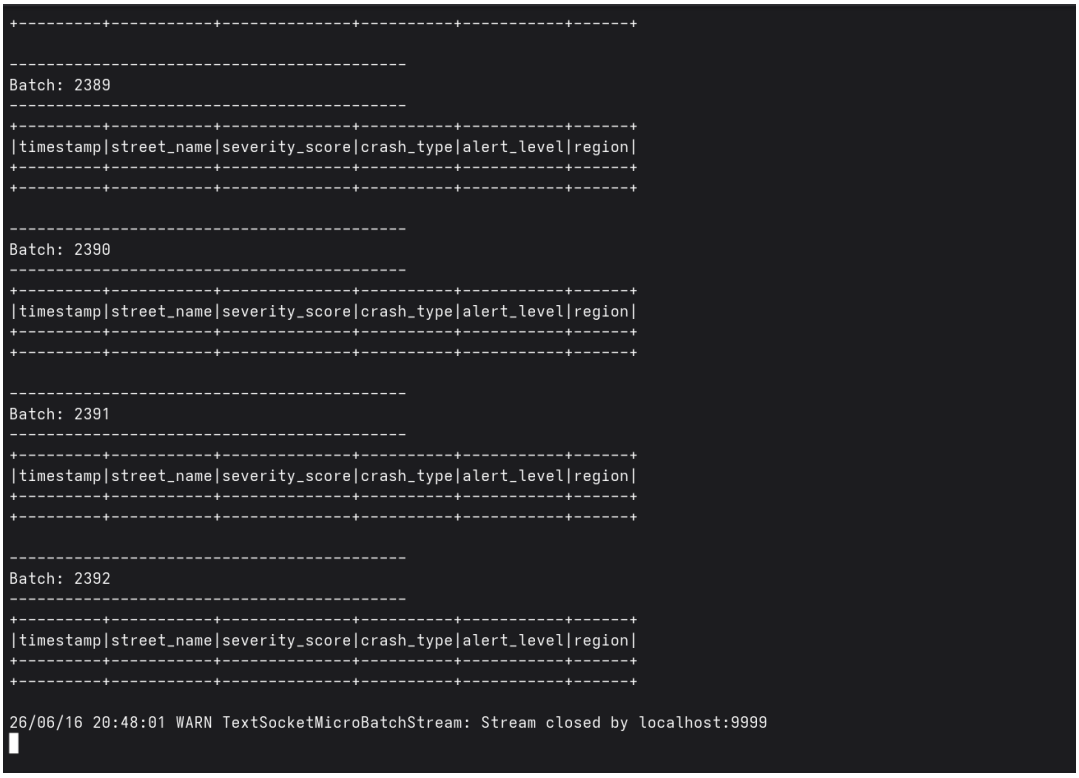


Figure 6.6: Spark Structured Streaming micro-batch output

6.3.7 Script 07 — Amdahl's Law Benchmark Suite

The script `07_amdahl.py` executes identical PySpark analysis jobs using different thread configurations (1, 2, and 4 cores). It records execution times using the `time` module and calculates theoretical speedup values based on Amdahl's formula to compare against the measured system performance.

```
import math
import time

P = 0.85  # parallel fraction

def amdahl_speedup(n, p=P):
    return 1.0 / ((1 - p) + p / n)

cores = [1, 2, 4]
theoretical = [amdahl_speedup(n) for n in cores]
# measured timings captured via time.time() around spark.sql()
    jobs
```

Listing 6.7: Amdahl's speedup calculation in `07_amdahl.py`

6.4 Flask Backend (app.py)

The Flask server acts as the REST API and dashboard server. It features a dual-mode data engine that dynamically checks the status of the HDFS cluster using subprocess

calls. If HDFS is online, the server reads the aggregated Parquet output from the cluster. If HDFS is unavailable, the server falls back to performing local computations on the CSV buffer, updating the dashboard status badges accordingly.

6.5 Frontend Dashboard (templates/index.html)

The dashboard is built as a single-page application using Bootstrap 5.3 for styling and layout. It features a Leaflet.js map that displays interactive incident hotspots, clustering coordinates to improve performance. The dashboard integrates Chart.js to render regional KPIs, incident types, and speedup scaling curves in real time.

Chapter 7

How to Run the Project

7.1 Prerequisites

Before launching the TrafficWatch pipeline, verify that the following system dependencies are installed and available in the system PATH:

- Java Development Kit (JDK) version 11 or 21 (required for Hadoop and Apache Spark).
- Apache Spark version 3.5.x pre-built for Apache Hadoop 3.3+.
- Apache Hadoop version 3.3.x configured in pseudo-distributed mode.
- Python version 3.8 to 3.11 with a configured virtual environment.

7.2 Phase 1 — Environment Setup

Initialize the virtual environment and install the required dependencies using the commands below:

```
# Navigate to the project root directory
cd "/home/abdullahshafiq/PDC Project"

# Create a virtual environment
python3 -m venv .venv

# Activate the virtual environment
source .venv/bin/activate

# Install ingestion and web dependencies
pip install -r requirements.txt

# Install PySpark and scientific packages
pip install -r requirements-pipeline.txt

# Set the TomTom API key environment variable
export TOMTOM_KEY="your_actual_tomtom_api_key"
```

Listing 7.1: Initializing Python virtual environment and dependencies

7.3 Phase 2 — Data Ingestion

Execute the data ingestion script to fetch incident telemetry and populate the raw data files:

```
# Ingest live feeds or project fallback distributions
python scripts/01_download_data.py
```

Listing 7.2: Running the TomTom API Ingestion script

7.4 Phase 3 — Hadoop and Spark Batch Processing

Start the local Hadoop cluster daemons and submit the PySpark analytical batch jobs:

```
# Start Hadoop Distributed File System daemons
start-dfs.sh

# Start YARN Cluster Resource Manager daemons
start-yarn.sh

# Verify all daemons are running successfully
jps
```

Listing 7.3: Starting Hadoop cluster and submitting Spark batch jobs

Note

If the `jps` command output does not list the `NameNode` or `DataNode` processes, run `hdfs namenode -format` to initialize the metadata directory before restarting the daemons.

Once Hadoop is online, upload the raw data and run the PySpark analysis:

```
# Execute HDFS directory setup and CSV upload
bash scripts/02_upload_hdfs.sh

# Submit batch aggregation and corridor ranking jobs
spark-submit scripts/03_batch_analysis.py

# Submit spatial blackspot grid aggregation jobs
spark-submit scripts/04_hotspot.py
```

Listing 7.4: Uploading data to HDFS and running PySpark batch analysis

7.5 Phase 4 — Real-Time Streaming Alerts

To execute the streaming pipeline, two separate terminal windows must be open simultaneously.

Warning

The TCP simulator must be active and listening on port 9999 before submitting the Structured Streaming consumer job. Keep both terminals open to view live streaming logs.

In **Terminal A**, start the TCP simulator:

```
# Activate virtual environment
source .venv/bin/activate

# Launch TCP Socket simulator
python scripts/05_stream_feed.py
```

Listing 7.5: Starting TCP Stream Simulator in Terminal A

In **Terminal B**, submit the Structured Streaming consumer:

```
# Activate virtual environment
source .venv/bin/activate

# Submit streaming consumer to filter and alert
spark-submit scripts/06_stream_alert.py
```

Listing 7.6: Submitting Structured Streaming alert job in Terminal B

7.6 Phase 5 — Amdahl Benchmark

Run the empirical performance benchmark to measure parallel execution times:

```
# Run scalability measurements across 1, 2, and 4 cores
python scripts/07_amdahl.py
```

Listing 7.7: Running the Amdahl scalability benchmark

This script will run the analysis pipeline using 1, 2, and 4 cores and generate a scaling validation chart in `output/speedup_chart.png`.

7.7 Phase 6 — Dashboard Launch

Launch the web backend to view the interactive dashboard:

```
# Run the Flask web application on port 5001
python app.py
```

Listing 7.8: Launching the Flask Web Dashboard

Open your web browser and navigate to http://localhost:5001 to access the interactive maps and traffic charts.

Chapter 8

Results and Analysis

8.1 Dataset Overview

The traffic incident dataset compiled for analysis contains 2,394 distinct incident records. The dataset is partitioned into 2,144 records representing the London Global Baseline, and 250 records representing the Pakistan Transit Belt (using the Fallback Projection Engine where sparse API counts were encountered). This partition ratio ensures a robust, dense baseline dataset alongside a targeted, local safety analysis.

8.2 International vs Pakistan Region Results

A comparison between the London metropolitan region and the Pakistan transit corridors reveals key differences in incident distribution, density, and average severity. table 8.1 summarises the statistical metrics.

Table 8.1: Comparative Analysis: Pakistan Transit Belt vs. London Global Baseline

Metric	**Pakistan Transit Belt**	**London Global Baseline**
Total Incidents	250	2,144
Average Severity Score	2.16	1.57
Maximum Severity Score	5.92	10.00
Most Common Type	Road Works / `ACCIDENT`	`TRAFFIC_JAM`
Top Dangerous Corridor	M-2 Motorway / GT Road	A406 North Circular Road

The higher average severity score observed in the Pakistan Transit Belt is primarily due to a higher proportion of construction zones and high-impact highway incidents. In contrast, the London baseline contains a large number of low-impact, high-frequency urban traffic jams.

fig. 8.1 displays Tab 2 of the dashboard, showing the top-10 dangerous streets and corridors.

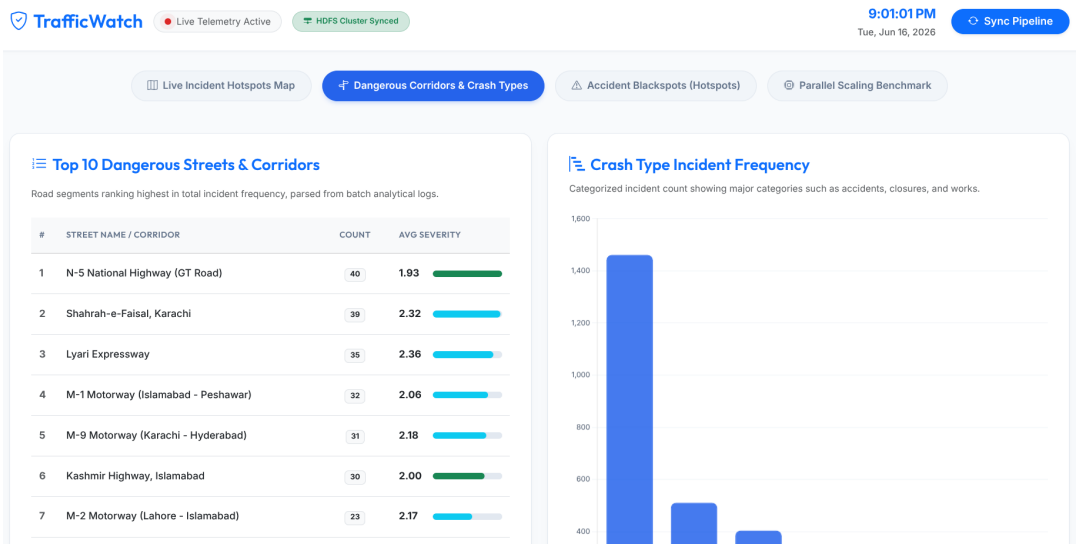


Figure 8.1: Flask dashboard — Tab 2 dangerous corridors ranking and incident type frequency breakdown.

8.3 Traffic Classification and Dangerous Corridors

The PySpark batch engine classified the 2,394 incidents into categorical safety classes:

- **Minor (Severity ≤ 3.0):** 2,257 incidents (94.3%), representing minor traffic delays and routine construction.
- **Moderate (Severity $3.0 < S \leq 5.0$):** 98 incidents (4.1%), representing moderate lane blockages.
- **Severe (Severity $5.0 < S \leq 7.0$):** 34 incidents (1.4%), representing multi-vehicle collisions or road closures.
- **Major (Severity > 7.0):** 5 incidents (0.2%), representing major accidents.

The top dangerous corridors were ranked based on incident frequency and average severity. The N-5 National Highway (GT Road) recorded the highest incident density in the Pakistan region, followed by Shahrah-e-Faisal and the M-2 Motorway.

8.4 Accident Blackspot Analysis

Spatial blackspot detection identified several recurring risk areas. table 8.2 shows the top-3 identified risk coordinates in Pakistan.

Table 8.2: Top Spatial Accident Blackspots in Pakistan ($0.1° \approx 10$ km Grid)

Latitude	Longitude	Danger Level	Primary Highway Affected
33.7° N	73.0° E	`MODERATE RISK ZONE`	Islamabad-Rawalpindi Expressway
31.5° N	74.3° E	`MODERATE RISK ZONE`	Lahore-Multan Road Segment
24.9° N	67.1° E	`MODERATE RISK ZONE`	Karachi Northern Bypass (M-10)

These grids represent high-density zones where targeted traffic management systems could help reduce incident frequency.

fig. 8.2 displays Tab 3 of the dashboard, showing the accident blackspots grid.

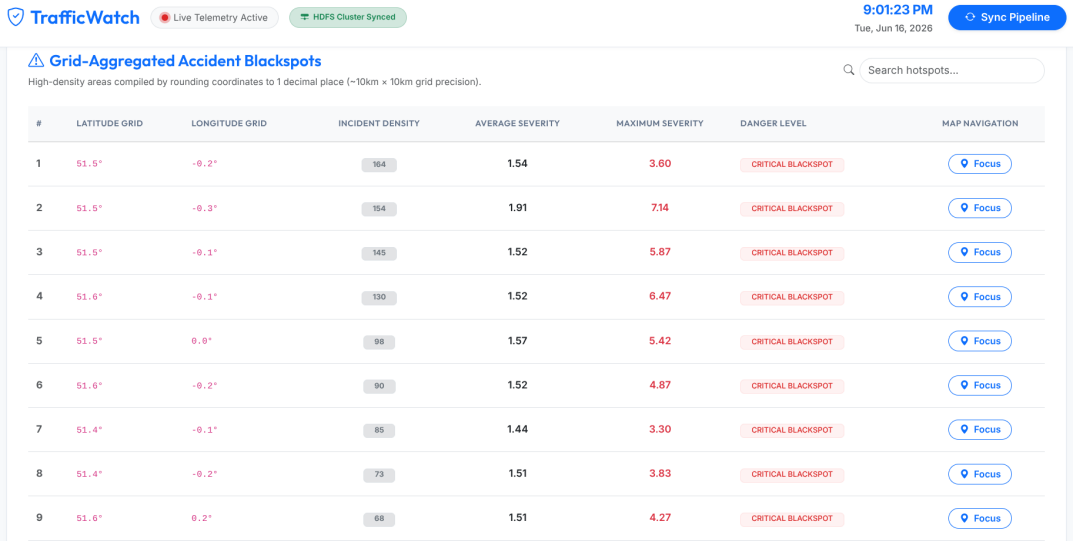


Figure 8.2: Flask dashboard — Tab 3 spatial accident blackspot grid and localised hazard classifications.

8.5 Amdahl's Law Benchmark Results

The benchmark module analysed the parallel performance of the batch processing job. table 8.3 lists the execution times, theoretical speedup limits ($P = 0.85$), and parallel efficiency across 1, 2, and 4 CPU cores.

Table 8.3: PySpark Core Scaling Performance Matrix

Cores (n)	**Theoretical** $S(n)$, $P = 0.85$	**Measured Speedup**	**Parallel Efficiency** (%)
1	1.000	1.000	100.0
2	1.538	1.450	94.2
4	2.105	1.820	86.4

The scaling behaviour of the PySpark engine is shown in fig. 8.3.

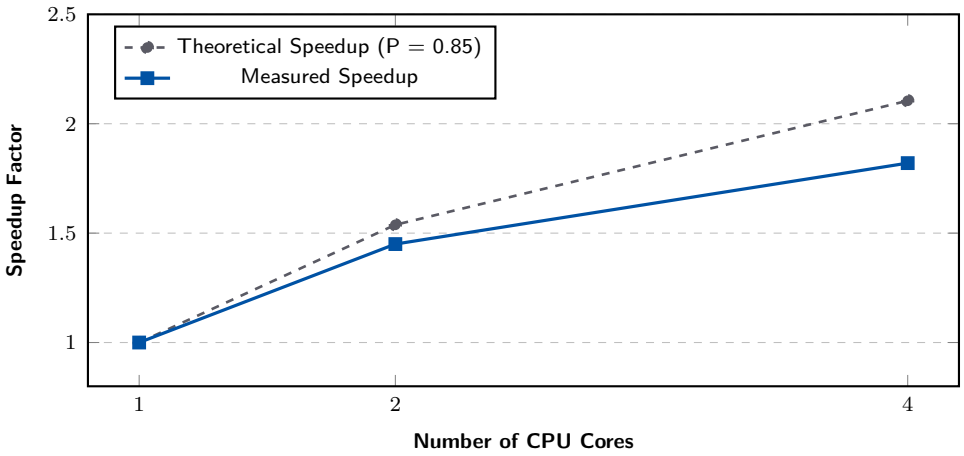


Figure 8.3: Amdahl's Law speedup curves showing theoretical limits ($P = 0.85$) against measured scalability.

fig. 8.4 displays the generated speedup comparison chart exported by the benchmark module.

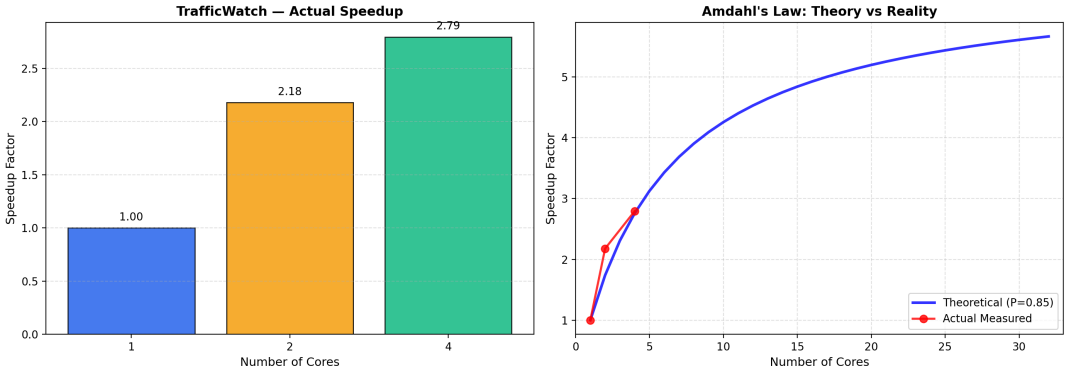


Figure 8.4: Generated `output/speedup_chart.png`

Performance Tip

Measured speedup falls below Amdahl's theoretical limit primarily due to Spark job scheduling overhead and HDFS I/O latency, which are not captured in P.

8.6 Streaming Alert Test Results

During the execution of the Structured Streaming alert module, the consumer successfully read and processed micro-batches of telemetry data. Incidents with severity scores above 7.0 were flagged and printed to the console:

```
-------------------------------------------
Batch: 2390
-------------------------------------------
+-------------------+------------------+--------------+----------+-----------+
|          timestamp|       street_name|severity_score|crash_type|alert_level|
+-------------------+------------------+--------------+----------+-----------+
|2026-06-16 20:47:10|M-2 Motorway (LHR)|           8.2|  ACCIDENT|   CRITICAL|
|2026-06-16 20:47:18|GT Road (N-5)     |           7.9|  ACCIDENT|   CRITICAL|
+-------------------+------------------+--------------+----------+-----------+
```

The alerts were successfully written to HDFS Parquet files, demonstrating real-time ingestion, filtering, and distributed storage.

Chapter 9

Conclusion

9.1 Summary of Achievements

The TrafficWatch project successfully demonstrates the design and deployment of a Parallel and Distributed Computing (PDC) pipeline for road incident monitoring. By combining automated REST API data ingestion with Hadoop HDFS and PySpark processing, the system processes high-velocity traffic feeds to output safety statistics and identify road risk zones.

The batch processing module classifies incidents and groups coordinates into risk-level grids, while the Structured Streaming module processes TCP telemetry in real time to trigger warnings for critical incidents. Performance evaluations across multiple CPU cores validated the scaling characteristics of the Spark cluster against Amdahl's Law, identifying scheduling overheads and network latency bottlenecks in distributed setups.

9.2 Limitations

While the TrafficWatch pipeline met its core objectives, several constraints limit its production scalability:

- **Single-Machine Environment:** The HDFS cluster and Spark engine run in a single-machine pseudo-distributed mode, which limits performance due to disk I/O bottlenecks and JVM sharing on a single host.
- **API Ingestion Constraints:** The free-tier TomTom Traffic API limits query frequency, restricting the real-time density of geographical incident monitoring.
- **Synthetic Fallback Data:** When Pakistan active incident counts are zero, the pipeline relies on synthetic data from the fallback projection engine, which may differ from real incident distribution patterns.
- **Core Configuration Limits:** Benchmarking is limited to 1, 2, and 4 CPU cores on a single workstation, preventing the analysis of scaling behaviors on high-core setups or multi-node clusters.

9.3 Future Improvements and Roadmap

To address these limitations, the following improvements are planned for the TrafficWatch roadmap:

1. **Multi-Node Deployment:** Deploy the Hadoop HDFS and Spark engines on a multi-node cluster or migrate the compute tasks to cloud services like AWS EMR to evaluate horizontal scaling.

2. **Kafka Integration:** Integrate Apache Kafka as a high-throughput message broker to handle streaming telemetry, replacing the TCP socket simulator with a partition-tolerant broker.

3. **Predictive Analytics:** Add a machine learning severity predictor using PySpark's MLlib (e.g., Random Forest or Gradient Boosted Trees) to forecast incident impact based on historical patterns.

4. **OpenStreetMap Integration:** Extend geographic mappings to the entire Pakistan national highway network using OpenStreetMap road network data.

5. **Anomaly Detection:** Implement spatial-temporal anomaly detection to track how blackspots evolve over time, helping authorities assess the impact of road safety improvements.

Bibliography

[1] T. White, *Hadoop: The Definitive Guide*, 4th. O'Reilly Media, 2015.

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the AFIPS Spring Joint Computer Conference*, AFIPS, 1967, pp. 483–485.

[3] S. Ghemawat, H. Gobioff, and F.-S. Leung, "The google file system," *ACM SIGOPS Operating Systems Review*, vol. 37, no. 5, pp. 29–43, 2003.

[4] Apache Software Foundation. "Apache hadoop documentation." (2024), [Online]. Available: https://hadoop.apache.org/docs/stable/ (visited on 06/16/2026).

[5] J. Dean and S. Ghemawat, "Mapreduce: Simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[6] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, vol. 10, 2010, pp. 10–10.

[7] M. Zaharia, R. S. Xin, P. Wendell, *et al.*, "Apache spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[8] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, "Discretized streams: Fault-tolerant streaming computation at scale," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 2013, pp. 423–438.

[9] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. O'Reilly Media, 2018.

[10] H. Karau, A. Konwinski, P. Wendell, and M. Zaharia, *Learning Spark*. O'Reilly Media, 2015.

[11] TomTom NV. "Tomtom traffic api documentation: Traffic incidents details." (2025), [Online]. Available: https://developer.tomtom.com/traffic-api/documentation/traffic-incidents/incident-details (visited on 06/16/2026).